

Contents

Stack Overflow Questions — Module 3 Lab Guide	2
The Scenario — Your Mission	2
Where you work	2
The problem	2
Your manager's request	3
Your team's job for the next 2 weeks (Module 3)	3
After Module 3	3
How This Guide Works (Read This First!)	3
We do NOT give you the full code	3
Visualizations — Two Modes	4
1. Exploratory charts (for YOU)	4
2. Explanatory charts (for TOM)	4
Your plotting toolkit	4
Before You Start — Google Colab Setup	4
Step 1 — Open a new Colab notebook	5
Step 2 — Connect to Google Drive	5
Step 3 — Create a project folder	5
Step 4 — Get the data	5
Step 5 — Sample 100,000 rows	5
Colab tips	6
Class 1 — Data Cleaning	6
Your goal	6
Inputs	6
Outputs	6
Phase A — Explore the data first (15 minutes)	6
Phase B — Clean the questions table (45 minutes)	7
Phase C — Make ONE chart for Tom (15 minutes)	8
Common mistakes in Class 1	8
Self-check before Class 2	9
Class 2 — Encoding and Scaling	9
Your goal	9
Inputs	9
Outputs	9
Phase A — Explore the data first	9
Phase B — Encode and scale	10
Phase C — Make ONE chart for Tom	11
Common mistakes in Class 2	11
Self-check before Class 3	11
Class 3 — Feature Engineering	12
Your goal	12
Inputs	12
Outputs	12
Phase A — Explore the data first	12
Phase B — Engineer the features	12
Phase C — Make ONE chart for Tom	13
Common mistakes in Class 3	14
Self-check before Class 4	14
Class 4 — Feature Selection	14

Your goal	14
Inputs	14
Outputs	14
Phase A — Explore the data first	14
Phase B — Select features	15
Phase C — Make ONE chart for Tom	16
Common mistakes in Class 4	16
Self-check before Class 5	16
Class 5 — Pipelines	16
Your goal	16
Inputs	16
Outputs	17
Phase A — Explore the data first	17
Phase B — Build the pipeline	17
Phase C — Make ONE chart for Tom	18
Common mistakes in Class 5	18
Self-check before Class 6	18
Class 6 — End-to-End Lab (THE BIG ONE)	19
Your goal	19
Time	19
Inputs	19
Outputs	19
Phase A — Explore the data first (10 minutes)	19
Phase B — Build the final dataset (70 minutes)	19
Phase C — Findings report for Tom (10 minutes)	20
Grading rubric (100 points)	21
Submit	21
Tips for the whole module	21

Stack Overflow Questions — Module 3 Lab Guide

Scenario: Programmer Q&A community. Predict if a question will get answered. **Module:** 3 (Data Preparation and Feature Engineering) **Audience:** Adult learners, A2 English. New to AI. **Tools:** Google Colab (no install needed).

The Scenario — Your Mission

Where you work

You and your team are new data analysts at **Stack Exchange**, the company that runs **Stack Overflow**. Stack Overflow is the biggest website where programmers ask questions and other programmers answer.

Every day: - **8,000 new questions** are posted. - Some get an answer in 5 minutes. - About **30% never get any answer**.

The problem

When a user posts a question and nobody answers, the user feels rejected. They never post again. The community loses members.

Tom is worried. He calls your team into a meeting.

Your manager's request

Your manager, **Tom** (Community Quality Manager), tells you:

“We lose 30% of new questions to silence. The user gets discouraged. They never come back.

I need a different tool. When a user is TYPING a draft question, we read it BEFORE they click submit.

If our model says ‘this question is unclear and will not get an answer’, we show a pop-up: ‘Your question is missing detail. Try adding sample code and what you have tried.’

If we save even 1 in 5 of these dying questions, we keep thousands of community members.”

Your team's job for the next 2 weeks (Module 3)

Tom shows you the raw data. It is one big CSV with about 100,000 questions. Each row has the title, body, tags, view count, score, and timing.

Your job in Module 3: > **Turn this messy CSV into ONE clean file. The clean file will be used to train the model in Module 4.**

The clean file is called `stackoverflow_questions_clean.parquet`. It must have **21 specific columns** (we will see them in Class 6).

After Module 3

Module	What happens
Module 4	Train the prediction model. Tom finally gets his “no-answer risk score”.
Module 5	Find groups of similar questions. Detect possible duplicates.
Module 7	Use the title and body text. Predict tags from the text alone.

You use the **same Stack Overflow dataset** until the end of Module 7.

How This Guide Works (Read This First!)

We do NOT give you the full code

In this guide you will see two kinds of content:

1. WHAT / WHY / EXPECTED — explained in words

For every step you read: - **WHAT** you have to do (in normal language) - **WHY** you do it (the reason) - **HINTS** — the function names, the arguments to remember - **EXPECTED** — the result you should see if you did it right

2. Sometimes a tiny code skeleton with blanks

For the harder steps we give you a code skeleton with ____ blanks. **You fill in the blanks.**

Why no full code?

Because you will NOT learn if you copy-paste. - In Module 4, the model only works if YOU understand each step. - In your future job, nobody will give you the code. - Writing the code yourself takes 10 minutes more today, but saves 10 hours later.

Rule: If you finish a step in 30 seconds by copy-pasting from a teammate's notebook, do it AGAIN by yourself. Different variable names, same logic.

Visualizations — Two Modes

In every class you will make charts. There are **two reasons** to make a chart:

1. Exploratory charts (for YOU)

Made BEFORE you clean the data. To understand what the data looks like. - Fast, ugly, no labels needed. - Examples: `df.hist()`, `df['column'].value_counts().plot.bar()`.

Goal: “**What does this data look like?**”

2. Explanatory charts (for TOM)

Made AFTER you understand. To EXPLAIN something to your manager. - Clean, labeled, one clear message. - Must have: title, x-label, y-label, source, and a 1-sentence takeaway.

Goal: “**Tom, look at this. This is the problem.**”

In every class you make BOTH kinds.

Your plotting toolkit

Plot	When to use it	Function name
Histogram	See the SHAPE of a numeric column	<code>plt.hist()</code> or <code>sns.histplot()</code>
Bar chart	Count of each category	<code>df['col'].value_counts().plot.bar()</code>
Box plot	See outliers in a numeric column	<code>sns.boxplot()</code>
Scatter plot	See the relationship between 2 numbers	<code>plt.scatter()</code>
Heatmap	See correlation between many columns	<code>sns.heatmap(df.corr())</code>
Line plot	See change over time	<code>plt.plot()</code>

Before you start: `import matplotlib.pyplot as plt` and `import seaborn as sns`.

Before You Start — Google Colab Setup

You will work in **Google Colab**. That means: - You do NOT install Python. - You do NOT install pandas, scikit-learn, or anything. - You just open a notebook in your web browser.

Step 1 — Open a new Colab notebook

1. Go to <https://colab.research.google.com>
2. Sign in with your Google account.
3. Click “**New notebook**” (top left).
4. Name it `stackoverflow_module_3.ipynb`.

Step 2 — Connect to Google Drive

Colab files **disappear** when you close the browser. So you must save your work to **Google Drive**.

In the first cell:

```
from google.colab import drive
drive.mount('/content/drive')
```

A pop-up asks for permission. Click “Allow”. After this, Drive is available at `/content/drive/MyDrive/`.

Step 3 — Create a project folder

In a new cell:

```
import os
os.makedirs('/content/drive/MyDrive/stackoverflow_lab', exist_ok=True)
%cd /content/drive/MyDrive/stackoverflow_lab
```

Your team will save ALL files in this folder.

Step 4 — Get the data

Option A — Direct from Kaggle (recommended):

1. Make a free Kaggle account.
2. Go to kaggle.com/settings/account, scroll to “API”, click “Create New Token”. Save the `kaggle.json` file.
3. In Colab:

```
from google.colab import files
files.upload() # pick kaggle.json
```

4. Then:

```
!mkdir -p ~/.kaggle && mv kaggle.json ~/.kaggle/ && chmod 600 ~/.kaggle/kaggle.json
!pip install kaggle -q
!kaggle datasets download -d stackoverflow/stackoverflow
!unzip -q stackoverflow.zip -d data
!ls data/
```

You should see the **CSV files** inside `data/`.

Option B — Upload by hand:

1. Download the zip from kaggle.com/datasets/stackoverflow/stackoverflow on your laptop.
2. Unzip it.
3. In Colab’s file panel (left sidebar), upload the questions file.

Step 5 — Sample 100,000 rows

The full dataset has 10 MILLION rows. Your laptop CANNOT load that. Sample 100,000:

```
import pandas as pd
df = pd.read_csv('data/questions.csv', nrows=100_000)
print(df.shape)
```

Should print (100000, ____). You are ready.

Colab tips

Tip	Why
Save your notebook to Drive often	Colab disconnects after 12 hours.
Save intermediate .parquet files to Drive	Files in /content/ disappear when Colab disconnects.
Share the notebook with teammates (Share button, top right)	Editor access. Like Google Docs for code.
Restart runtime if memory full	Runtime > Restart runtime.

Class 1 — Data Cleaning

Scenario reminder: Tom drops a 10-million-row CSV on your desk. You cannot load it all. Some columns are HTML. Some date columns are stored as text. Your job today: sample, clean, and prepare the basic table.

Your goal

Sample the data. Fix the HTML in the body column. Convert dates. Decide what to keep.

Inputs

- The raw CSV from Kaggle in data/

Outputs

- questions_step1.parquet saved in your Drive folder
- 3+ exploratory charts in your notebook
- 1 explanatory chart for Tom
- Notes in markdown about every decision you made

Phase A — Explore the data first (15 minutes)

Before you clean anything, **LOOK** at the data.

Exploratory chart 1 — How many questions per year?

- **Question:** “Is the dataset balanced across years? Or do we have mostly recent questions?”
- **HINTS:**
 - First convert `creation_date` to `datetime`.
 - Use `.dt.year` to get the year.
 - `value_counts()` then `.sort_index().plot.bar()`.
- **What you learn:** Time distribution. Are old questions over-represented?

Exploratory chart 2 — Distribution of answer_count

- **Question:** “Most questions get 1 answer. Some get 0. A few get 50+. What is the shape?”
- **HINTS:**
 - Histogram with `bins=30`.
 - Add `range=(0, 20)` to ignore the long tail.
- **What you learn:** The long tail. The 0-answer questions are the main problem.

Exploratory chart 3 — Top 20 most common tags

- **Question:** “Which programming languages are most popular?”
 - **HINTS:**
 - The tags column is a string like `<python><pandas><dataframe>`.
 - Use a regex `re.findall(r'<(.*?)>', tags_string)` to extract.
 - Flatten, count, top 20.
 - **What you learn:** Python and JavaScript dominate. Other languages are smaller.
-

Phase B — Clean the questions table (45 minutes)

Step 1 — Load with row limit

- **WHAT:** Load only 100,000 rows with `pd.read_csv()`.
- **HINTS:**
 - Use `nrows=100_000`.
 - Set `low_memory=False`.
- **EXPECTED:** A DataFrame with about 100,000 rows x ~10 columns.

Step 2 — Look at the DataFrame

- **WHAT:** Run `.info()`, `.head()`, `.shape`, and `.dtypes`.
- **HINTS:**
 - Look at the Dtype column in `.info()` output. **Is creation_date stored as object (text)?**
 - Look at body — it has HTML tags like `<p>`, `<code>`, `<pre>`.
- **EXPECTED:** ~100,000 rows. Date columns are text. Body is full of HTML.

Step 3 — Convert dates

- **WHAT:** Convert `creation_date` and any other date column to datetime.
- **HINTS:**
 - Use `pd.to_datetime()` with `errors='coerce'`.
 - `errors='coerce'` means: if a cell is bad, set it to NaT (Not a Time = missing).
- **WHY:** If dates are strings, you cannot compute “how many days ago?”.
- **EXPECTED:** `df['creation_date'].dtype` is `datetime64[ns]`.

Step 4 — Strip HTML from body

- **WHAT:** The body column has `<p>...</p>`, `<code>...</code>`, `<a href="...">...</a>` tags.
- **HINTS:**
 - Use BeautifulSoup: `from bs4 import BeautifulSoup`.
 - Make a function: `def strip_html(text): return BeautifulSoup(text, 'html.parser').get_text()`.
 - Apply to body: `df['body_clean'] = df['body'].apply(strip_html)`.
 - WARNING: BeautifulSoup is slow. For 100k rows, it takes ~30 seconds. That is OK.
- **WHY:** TF-IDF in M7 should see real WORDS, not HTML tags.

Step 5 — Count code blocks BEFORE stripping HTML

Code blocks are a STRONG signal of quality. Save the count before you strip.

- **WHAT:** Count `<pre><code>` tags per row.
- **HINTS:**
 - Use `df['body'].str.count('<pre>')`. Or `<code>`. Both work.
 - Save as `n_code_blocks`.
- **WHY:** Questions with code are more likely to be answered.

Step 6 — Find missing values

- **WHAT:** Run `df.isna().sum()`.
- **EXPECTED:** Some rows have empty body or empty tags. Decide what to do.

Step 7 — Drop rows with empty body or empty tags

- **WHAT:** Use `.dropna(subset=['body', 'tags'])`.
- **WHY:** We cannot work with no content.

Step 8 — Write down what you did

In a markdown cell, write: - Starting rows: ~100,000. - Rows after dropping empty body / tags: ~_____. - WHY you removed each group.

Step 9 — Save to Drive

- **WHAT:** Save the cleaned DataFrame as parquet.
 - **HINTS:** `df.to_parquet('/content/drive/MyDrive/stackoverflow_lab/questions_step1.parquet')`.
-

Phase C — Make ONE chart for Tom (15 minutes)

He does not have time to read your code. He wants ONE picture.

Tom's chart — “The silent question problem”

Make a bar chart showing two bars: - Questions with at least 1 answer - Questions with 0 answers

- **HINTS:**
 - `is_answered = (df['answer_count'] > 0).`
 - `is_answered.value_counts().plot.bar()`.
 - Add the title: "30% of questions get NO answer. That is our problem."
 - **Takeaway for Tom:** “We must catch the silent ones BEFORE they are posted.”
-

Common mistakes in Class 1

Mistake	What goes wrong
Load all 10 million rows	Colab runs out of memory and crashes.
Forget <code>errors='coerce'</code> on <code>to_datetime</code>	One bad date crashes the code.
Strip HTML with simple regex	Misses nested tags. Use BeautifulSoup.
Save to <code>/content/</code> instead of Drive	File disappears when Colab disconnects.

Self-check before Class 2

- ☐ You sampled 100,000 rows.
 - ☐ Date columns have dtype `datetime64`.
 - ☐ You have a `body_clean` column with no HTML.
 - ☐ You counted `n_code_blocks` BEFORE stripping HTML.
 - ☐ At least 3 exploratory charts in your notebook.
 - ☐ 1 polished chart for Tom.
 - ☐ You saved `questions_step1.parquet` to your Drive folder.
-

Class 2 — Encoding and Scaling

Scenario reminder: Tom is happy with your cleaning. But he asks: “the tags column is one big string like `<python><pandas>`. The `view_count` ranges from 1 to 1,000,000. The model is a math model. Make these usable.”

Your goal

Turn the multi-value tags into proper columns. Make all numeric columns about the same size.

Inputs

- `questions_step1.parquet` from Class 1

Outputs

- `questions_step2.parquet` in Drive
 - 3+ exploratory charts
 - 1 explanatory chart for Tom
-

Phase A — Explore the data first

Exploratory chart 1 — Distribution of `view_count`

- **Question:** “Most questions get 100 views. A few get 1 million. Is the distribution skewed?”
- **HINTS:**
 - Use a histogram (`plt.hist()`).
 - Use `bins=50` to see the shape.
- **What you learn:** Very long tail. We will use log-transform.

Exploratory chart 2 — Distribution of `view_count` AFTER `np.log1p`

- **HINTS:**
 - Make `view_count_log = np.log1p(df['view_count'])`.
 - Histogram it.
 - Compare to chart 1.
- **What you learn:** Log makes the long tail manageable.

Exploratory chart 3 — Top 20 most common tags

- **HINTS:**
 - Already done in Class 1 — repeat here for completeness.

- `value_counts().head(20).plot.bar()`.
- **What you learn:** Python, JavaScript, Java, C# dominate. Smaller languages are rare.

Exploratory chart 4 — `n_code_blocks` vs `is_answered`

- **HINTS:**
 - Group by `is_answered`. Take mean of `n_code_blocks`.
 - Bar chart.
 - **What you learn:** Answered questions have MORE code blocks on average.
-

Phase B — Encode and scale

Step 1 — Parse the tags column

- **WHAT:** The tags column is text: `<python><pandas><dataframe>`. Convert to a list.
- **HINTS:**
 - Use `re.findall(r'<(.*?)>', tag_string)`.
 - Apply to each row: `df['tag_list'] = df['tags'].apply(lambda x: re.findall(r'<(.*?)>', x))`.
- **EXPECTED:** A new column `tag_list` with Python lists like `['python', 'pandas', 'dataframe']`.

Step 2 — Count tags per row

- **WHAT:** Add a column `n_tags`.
- **HINTS:**
 - `df['n_tags'] = df['tag_list'].apply(len)`.
- **EXPECTED:** Values 1 to 5 (Stack Overflow limits to 5).

Step 3 — Extract the primary tag

- **WHAT:** The FIRST tag is usually the most important (the language).
- **HINTS:**
 - `df['primary_tag'] = df['tag_list'].apply(lambda x: x[0] if len(x) > 0 else 'unknown')`.

Step 4 — One-hot encode the top 20 tags

- **WHAT:** Make 20 new 0/1 columns for the most common tags.
- **HINTS:**
 - Find the top 20 tags from the exploration.
 - For each tag, add a column: `df[f'has_{tag}'] = df['tag_list'].apply(lambda x: 1 if tag in x else 0)`.
- **WHY:** A question tagged python has a different answer pattern than one tagged c++.

Step 5 — Log-transform `view_count`, `score`, `comment_count`

- **WHAT:** All three have long tails.
- **HINTS:**
 - `df['view_count_log'] = np.log1p(df['view_count'])`.
 - Same for `score` and `comment_count`.
- **WHY:** Long tails confuse models. Log fixes this.

Step 6 — Date-derived features

Make these from `creation_date`:

New column	How to make it
<code>creation_year</code>	<code>df['creation_date'].dt.year</code>
<code>creation_month</code>	<code>.dt.month</code>
<code>creation_dayofweek</code>	<code>.dt.dayofweek (0=Monday)</code>
<code>creation_hour</code>	<code>.dt.hour</code>
<code>is_weekend</code>	<code>(creation_dayofweek >= 5).astype(int)</code>

Step 7 — Save

- **HINTS:** `df.to_parquet('/content/drive/MyDrive/stackoverflow_lab/questions_step2.parquet')`.

Phase C — Make ONE chart for Tom

Tom's chart — “Hour of day vs answer rate”

- **HINTS:**
 - `GroupBy creation_hour`. Take the mean of `is_answered` (after Class 3 you will have this column; for now use `(answer_count > 0)`).
 - Line plot.
- **Title:** “When you ask the question matters — questions posted at 3am rarely get answers.”
- **Takeaway for Tom:** “Maybe show a warning to users posting at unusual hours.”

Common mistakes in Class 2

Mistake	What goes wrong
One-hot encode ALL tags	Some tags appear once. Adding 50,000 columns crashes the model.
Scale BEFORE the train/test split	Leakage.
Forget <code>np.log1p</code> and use <code>np.log</code> on zeros	<code>np.log(0) = -inf</code> . Crash.
Confuse the tags string with a Python list	The raw column is text. Must parse first.

Self-check before Class 3

- ☐ `tag_list` exists (Python list per row).
- ☐ `primary_tag` exists.
- ☐ Top-20 tags are one-hot encoded.
- ☐ `view_count`, `score`, `comment_count` are log-transformed.
- ☐ Date features extracted.
- ☐ 3+ exploratory charts.
- ☐ 1 chart for Tom.
- ☐ `questions_step2.parquet` saved to Drive.

Class 3 — Feature Engineering

Scenario reminder: Tom says: “Now make me the SMART columns. Length of the title. Length of the body. Does the body have an error message? Does it look like a ‘do my homework’ question? These are the signals.”

Your goal

Engineer signals from the title and body. These predict whether a question will be answered.

Inputs

- `questions_step2.parquet`

Outputs

- `questions_step3.parquet` in Drive
 - 3+ exploratory charts
 - 1 explanatory chart for Tom
-

Phase A — Explore the data first

Exploratory chart 1 — `is_answered` distribution

- First make the `is_answered` column (Step 1 below).
- **HINTS:** `df['is_answered'].value_counts(normalize=True).plot.bar()`.
- **What you learn:** Confirm it is around 70% Yes, 30% No.

Exploratory chart 2 — Title length vs answered rate

- **HINTS:**
 - Bin `title_length` into 10 bins with `pd.cut()`.
 - GroupBy bin, take mean of `is_answered`.
 - Bar chart.
- **What you learn:** Very short titles (1-3 words) and very long titles (>20 words) tend to NOT get answers.

Exploratory chart 3 — Body length vs answered

- **HINTS:** Same as above but for `body_length`.
- **What you learn:** Very short bodies (no detail) rarely get answers.

Exploratory chart 4 — `n_code_blocks` vs answered

- **HINTS:** Bar chart of mean `is_answered` per `n_code_blocks` value (0, 1, 2, 3+).
 - **What you learn:** 0 code blocks = often no answer.
-

Phase B — Engineer the features

Step 1 — Create the target column `is_answered`

- **WHAT:** `is_answered = 1` if `answer_count > 0`, else 0.
- **HINTS:**
 - Compare: `df['answer_count'] > 0`.

- Convert to int with `.astype(int)`.
- **EXPECTED:** About 70% of rows have `is_answered = 1`.

Step 2 — Title and body lengths

- **WHAT:** Length in characters AND length in words.

New column	How to make it
<code>title_length</code>	<code>df['title'].str.len()</code>
<code>title_n_words</code>	<code>df['title'].str.split().apply(len)</code>
<code>body_length</code>	<code>df['body_clean'].str.len()</code>
<code>body_n_words</code>	<code>df['body_clean'].str.split().apply(len)</code>
<code>n_paragraphs</code>	<code>df['body_clean'].str.count('\n\n') + 1</code>

Step 3 — HTML feature signals (already extracted in Class 1)

Confirm you have: - `n_code_blocks` — count of `<pre><code>` blocks - `n_links` — count of `<a` tags in body (extract before HTML stripping if needed) - `n_images_in_body` — count of `<img` tags

Step 4 — Text quality signals

Make these from `title` and `body_clean`:

New column	What it is	Hint
<code>title_n_caps_words</code>	Count of ALL-CAPS words in title	<code>re.findall(r'\b[A-Z]{2,}\b', title)</code>
<code>title_has_question_mark</code>	1 if title ends with ?	<code>.endswith('?')</code>
<code>body_has_error_message</code>	1 if body contains words like “Error:”, “Traceback”, “Exception”	<code>.str.contains('Error: Traceback Exception', regex=True)</code>
<code>has_minimal_reproducible_example</code>	1 if body says “what I have tried” or “I tried” or “here is my code”	regex search
<code>body_has_url</code>	1 if body has a URL	regex <code>http\S+</code>

- **WHY:** These are quality signals. Questions with error messages get more answers (the asker is specific).

Step 5 — Save

- **HINTS:** `df.to_parquet('/content/drive/MyDrive/stackoverflow_lab/questions_step3.parquet')`.

Phase C — Make ONE chart for Tom

Tom’s chart — “Code blocks predict answers”

A bar chart showing answer rate for: 0 code blocks, 1, 2, 3+.

- **HINTS:**
 - `df.groupby('n_code_blocks')['is_answered'].mean().plot.bar()`
 - Cap at `n_code_blocks=3+` (group 3, 4, 5, ... together).
- **Title:** “Questions with 2+ code blocks are 40% more likely to be answered.”
- **Takeaway for Tom:** “Our pop-up should say: ‘Add a code example to improve your chances.’”

Common mistakes in Class 3

Mistake	What goes wrong
Use <code>answer_count</code> as a feature	Leakage. <code>answer_count > 0 = is_answered</code> .
Forget to remove HTML before counting words	Counts <code><p></code> and <code><a></code> as words.
Use simple <code>.split()</code> for word count on HTML	Each tag becomes a “word”.

Self-check before Class 4

- ☐ `is_answered` exists. Mean ~70%.
 - ☐ Title and body lengths computed.
 - ☐ HTML signals (`n_code_blocks`, `n_links`, `n_images_in_body`) exist.
 - ☐ Text quality signals (`has_error_message`, etc.) exist.
 - ☐ 3+ exploratory charts.
 - ☐ 1 chart for Tom.
 - ☐ `questions_step3.parquet` saved to Drive.
-

Class 4 — Feature Selection

Scenario reminder: Now you have ~30 columns. Tom says: “Too many. Some are duplicates. Some are useless. I want 12-15 GOOD columns. Pick them.”

Your goal

Pick the best 12-15 columns. Drop the rest. Justify every choice.

Inputs

- `questions_step3.parquet`

Outputs

- `questions_step4.parquet` in Drive (only the selected columns + `is_answered`)
 - 3+ exploratory charts
 - A short markdown report explaining your choices
-

Phase A — Explore the data first

Exploratory chart 1 — Correlation heatmap

- **HINTS:**
 - `df[numeric_cols].corr()`.
 - `sns.heatmap(..., annot=True, fmt='.2f', cmap='coolwarm')`.
- **What you learn:** Pairs with `|corr| > 0.9` are redundant.

Exploratory chart 2 — Mutual information bars

- After Step 4 below, plot the result.
- **What you learn:** Which features predict `is_answered` strongest.

Exploratory chart 3 — Random Forest importance bars

- After Step 5 below, plot the result.
 - **What you learn:** A second opinion on importance.
-

Phase B — Select features

Step 1 — Split into train and test FIRST

- **WHAT:** Use `train_test_split`.
- **HINTS:**
 - `from sklearn.model_selection import train_test_split.`
 - `X = df.drop('is_answered', axis=1); y = df['is_answered'].`
 - Pass: `test_size=0.2, random_state=42, stratify=y.`
- **WHY:** All next steps use TRAIN ONLY. Otherwise leakage.

Step 2 — Remove low-variance columns

- **HINTS:**
 - `from sklearn.feature_selection import VarianceThreshold.`
 - `VarianceThreshold(threshold=0.01) then .fit(X_train[numeric_cols]).`

Step 3 — Remove highly correlated columns

- **HINTS:**
 - Compute correlation matrix.
 - Get the upper triangle with `np.triu(...)`.
 - For each column, if any of its upper-triangle correlations > 0.9 , mark for drop.

Step 4 — Rank by mutual information

- **HINTS:**
 - `from sklearn.feature_selection import mutual_info_classif.`
 - `mi = mutual_info_classif(X_train[numeric_cols], y_train).`
 - Sort and inspect.
- **EXPECTED:** Top features: `n_code_blocks`, `body_length`, `creation_hour`, `view_count_log`.

Step 5 — Random Forest importance

- **HINTS:**
 - `RandomForestClassifier(n_estimators=50, max_depth=8, n_jobs=-1).`
 - `.fit(X_train, y_train).`
 - Look at `.feature_importances_`.

Step 6 — Pick the final 12-15 columns

- **WHAT:** Combine the rankings. Pick columns that are:
 - High in mutual info, AND
 - High in RF importance, AND
 - Not dropped by variance/correlation pruning.
- **WRITE DOWN:** In your notebook, list the columns. Explain WHY for each.

Step 7 — Save

- Keep only selected columns + `is_answered`. Save as `questions_step4.parquet` to Drive.
-

Phase C — Make ONE chart for Tom

Tom's chart — “Top 10 predictors of an unanswered question”

A horizontal bar chart of the top 10 features by RF importance.

- **HINTS:**
 - `sort_values()` then `.head(10)` then `.plot.barh()`.
 - **Title:** “Top 10 predictors of question quality.”
 - **Takeaway for Tom:** “Code blocks and body length are the strongest signals. Our pop-up should focus on these.”
-

Common mistakes in Class 4

Mistake	What goes wrong
Compute mutual info on FULL data	Leakage.
Drop columns without writing why	Module 4 students will not understand.
Keep too many columns (40+)	Slow training. Overfitting risk.
Drop a high-mutual-info column	Big mistake. Check why before dropping.

Self-check before Class 5

- ☐ Train/test split done FIRST.
 - ☐ 12-15 columns remain + `is_answered`.
 - ☐ You wrote down WHY for each kept column.
 - ☐ 3+ exploratory charts.
 - ☐ 1 chart for Tom.
 - ☐ `questions_step4.parquet` saved to Drive.
-

Class 5 — Pipelines

Scenario reminder: Tom says: “Your cleaning code is in 4 different notebooks. Tomorrow when a user types a draft question on our website, will you copy 4 notebooks to the server? No. We need ONE object that does everything.”

Your goal

Put EVERY cleaning step inside ONE Pipeline object. Production-ready code.

Inputs

- `questions_step4.parquet`

Outputs

- `stackoverflow_pipeline.joblib` saved in Drive
 - 1 confusion-matrix + 1 ROC curve chart
-

Phase A — Explore the data first

Exploratory chart 1 — Confusion matrix

- After training, build a confusion matrix.
- **HINTS:**
 - `from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay.`
 - `ConfusionMatrixDisplay(cm, display_labels=['No answer', 'Answered']).plot().`

Exploratory chart 2 — ROC curve

- **HINTS:**
 - `from sklearn.metrics import RocCurveDisplay.`
 - `RocCurveDisplay.from_estimator(pipeline, X_test, y_test).`
-

Phase B — Build the pipeline

Step 1 — Decide which columns are numeric and which are categorical

- **HINTS:** Make two lists.
- **EXAMPLE:**
 - numeric: `view_count_log, score_log, comment_count_log, body_length, title_length, n_code_blocks, n_links, n_tags, creation_hour.`
 - categorical: `primary_tag, creation_dayofweek` (treat as categorical).

Step 2 — Build the numeric mini-pipeline

- **HINTS:**
 - Skeleton:

```
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='___')),
    ('scaler', StandardScaler()),
])
```
 - Fill in: best strategy for numeric? ('median')

Step 3 — Build the categorical mini-pipeline

- **HINTS:**
 - Skeleton:

```
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='___')),
    ('onehot', OneHotEncoder(handle_unknown='___', sparse_output=___)),
])
```
 - Fill in: 'most_frequent', 'ignore', False.
- **WHY `handle_unknown='ignore'`?** A new programming language tag might appear in production. Do not crash.

Step 4 — Combine into a ColumnTransformer

- **HINTS:**
 - `from sklearn.compose import ColumnTransformer.`
 - It takes a list of tuples: (name, transformer, list_of_columns).

Step 5 — Add the model on top

- **HINTS:**
 - `LogisticRegression(max_iter=1000, random_state=42).`
 - Class imbalance is mild (~70/30), but `class_weight='balanced'` is still safer.
- **WHY:** Without `class_weight='balanced'`, the model is biased toward the majority class.

Step 6 — Train and evaluate

- **HINTS:**
 - `full_pipeline.fit(X_train, y_train).`
 - `y_pred = full_pipeline.predict(X_test).`
 - `print(classification_report(y_test, y_pred)).`
- **EXPECTED:** F1 on the “No answer” class around 0.40-0.55.

Step 7 — Save the trained pipeline

- **HINTS:**
 - `import joblib.`
 - `joblib.dump(full_pipeline, '/content/drive/MyDrive/stackoverflow_lab/stackoverflow_pipeline.pkl')`

Phase C — Make ONE chart for Tom

Tom’s chart — “Of 1,000 unanswered questions, our baseline catches X”

A simple confusion matrix with labels.

- **HINTS:**
 - Use `seaborn.heatmap()` on the confusion matrix.
 - Annotate with the actual counts.
- **Title:** “Of 1,000 unanswered questions, we flag 550 correctly.”
- **Takeaway for Tom:** “Better than zero (which is what we have now). Module 4 will improve this.”

Common mistakes in Class 5

Mistake	What goes wrong
Build pipeline AFTER manual scaling	Pipeline scales twice. Wrong numbers.
Forget <code>sparse_output=False</code>	Some downstream code breaks.
Use <code>class_weight='balanced'</code> AND SMOTE	Over-correction. Pick one.
Forget <code>random_state</code>	Results change every run. Hard to debug.

Self-check before Class 6

- ☐ Pipeline has preprocessor + model.
- ☐ Preprocessor has numeric + categorical transformers.
- ☐ `handle_unknown='ignore'` set.

- ☐ `class_weight='balanced'` set.
- ☐ Confusion matrix + ROC curve charts.
- ☐ `stackoverflow_pipeline.joblib` saved to Drive.

Class 6 — End-to-End Lab (THE BIG ONE)

Scenario reminder: Tom is in the meeting room. He wants the FINAL clean dataset on his desk in 90 minutes. This is the lab.

Your goal

Take the raw CSV. Produce ONE final `.parquet` file with the exact 21-column schema. Plus a 1-page findings report.

Time

90 minutes of focused work.

Inputs

- The raw Stack Overflow CSV in your Drive folder

Outputs

- `stackoverflow_questions_clean.parquet` (~100,000 rows x 21 columns) in Drive
- `findings.md` (~1 page)
- Your full Colab notebook

Phase A — Explore the data first (10 minutes)

Reuse 3 of your best charts from Classes 1-5: 1. `is_answered` distribution 2. Code blocks vs answer rate 3. Top 10 most important features

These tell Tom the whole story.

Phase B — Build the final dataset (70 minutes)

Required output schema

#	Column	Type	Source
1	<code>question_id</code>	<code>int</code>	raw
2	<code>creation_year</code>	<code>int</code>	engineered
3	<code>creation_month</code>	<code>int</code>	engineered
4	<code>creation_dayofweek</code>	<code>int</code>	engineered
5	<code>creation_hour</code>	<code>int</code>	engineered
6	<code>is_weekend</code>	<code>int (0/1)</code>	engineered
7	<code>view_count_log</code>	<code>float</code>	engineered
8	<code>score_log</code>	<code>float</code>	engineered
9	<code>comment_count_log</code>	<code>float</code>	engineered
10	<code>title_length</code>	<code>int</code>	engineered

#	Column	Type	Source
11	body_length	int	engineered
12	n_code_blocks	int	engineered
13	n_links	int	engineered
14	n_images_in_body	int	engineered
15	has_error_message	int (0/1)	engineered
16	has_minimal_reproducible_example_phrase	int (0/1)	engineered
17	n_tags	int	engineered
18	primary_tag	string	engineered
19	n_paragraphs	int	engineered
20	is_answered	int (0/1)	TARGET (engineered)
21	body_clean	string	raw HTML-stripped (kept for Module 7)

Lab phases (timed)

Phase	Time	What you do
1. Load	10 min	Load CSV, sample 100k rows.
2. Clean	15 min	Parse dates, strip HTML, drop empty body/tags.
3. Encode	15 min	Parse tags, count tags, primary tag, log-transform numerics.
4. Engineer	20 min	Title/body lengths, HTML signals, text quality signals, dates.
5. Target	5 min	Make is_answered.
6. Validate + save	10 min	Check 21 columns + dtypes. Save .parquet.
7. Findings	10 min	Write findings.md.

Total: 90 minutes.

Phase C — Findings report for Tom (10 minutes)

Write findings.md with these sections:

Final numbers

- Final row count: _____
- is_answered rate: _____% (should be ~70%)

Top 3 insights

1. _____
2. _____
3. _____

One question to investigate in Module 4

- _____

One chart that summarizes everything

Embed your most important chart.

Grading rubric (100 points)

Item	Points
All 21 columns exist with the right names and dtypes	25
Cleaning decisions written in markdown	10
Pipeline reproducible (one command from raw CSV to .parquet)	15
BeautifulSoup stripping works on all HTML	10
is_answered rate is between 60% and 80%	10
At least 3 exploratory + 1 explanatory chart per class	15
findings.md has insights, not just numbers	10
Code is clean (comments, sensible variable names)	5

Submit

Upload to module-3/class_6/submissions/<YourTeamName>/: -stackoverflow_questions_clean.parquet
- Your Colab notebook (File > Download > Download .ipynb) - findings.md

Tips for the whole module

1. **Do NOT copy-paste code from this guide.** Write the code yourself.
2. **Save your notebook to Drive often.** Colab disconnects after 12 hours.
3. **Save intermediate .parquet files to Drive.**
4. **Pair-program.** One student types, one reads. Switch every 20 minutes.
5. **Make a chart BEFORE you write cleaning code.** You must SEE the problem before you can fix it.
6. **Make a chart AFTER each step.** Confirm your code did what you expected.
7. **Ask the mentor early.** If you are stuck for 20 minutes, ask. Do not waste an afternoon.

Good luck.